Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	27.03.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

1. Functional Requirements

Functional requirements define **what** the system should do to meet the business need for mandatory data.

- **Field Enforcement:** The system must evaluate specific conditions (e.g., *State is Resolved*) and dynamically set specified fields to "Mandatory."
- **Real-Time Feedback:** Validation must occur on the client side immediately when a user changes a field value, without requiring a page reload or form submission.
- **Conditional Logic:** The policy must support "Reverse if False" logic, ensuring that if conditions are no longer met, the mandatory requirement is automatically lifted.
- **UI Policy Actions:** The solution must include specific UI Policy Actions linked to the parent policy to target individual fields (e.g., `short_description`, `assignment_group`).
- **Conflict Resolution:** The UI Policy must be assigned an **Order** (e.g., 100, 200) to ensure it executes in the correct sequence relative to other policies on the same table.

2. Non-Functional Requirements

Non-functional requirements define **how** the system performs and its long-term maintainability.

- **Performance:** UI Policies must execute in under **100ms** to ensure no perceptible lag in the user interface.
- **Usability:** Mandatory fields must be visually highlighted (typically with a red asterisk) to guide the user.
- **Scalability:** The logic should be table-specific but written such that it doesn't interfere with extended tables unless explicitly intended.
- **Maintainability:** Use the "Condition Builder" rather than custom scripting (Execute if True/False) whenever possible to allow non-developers to manage the logic.
- **Data Integrity:** While UI Policies enforce mandatory fields on the form, a **Data Policy** should be considered if the fields must also be mandatory via Import Sets or Web Services.

3. Migration Requirements (Update Sets)

To ensure the changes move safely from Development to Production, the following migration standards must be met.

Capture Requirements

- **Scoped Selection:** All UI Policies and their associated UI Policy Actions must be captured in the **same** active Update Set.
- **Naming Convention:** Update Sets must follow a standard naming convention (e.g., `DEV-101_Mandatory_Fields_v1`) for easy tracking.

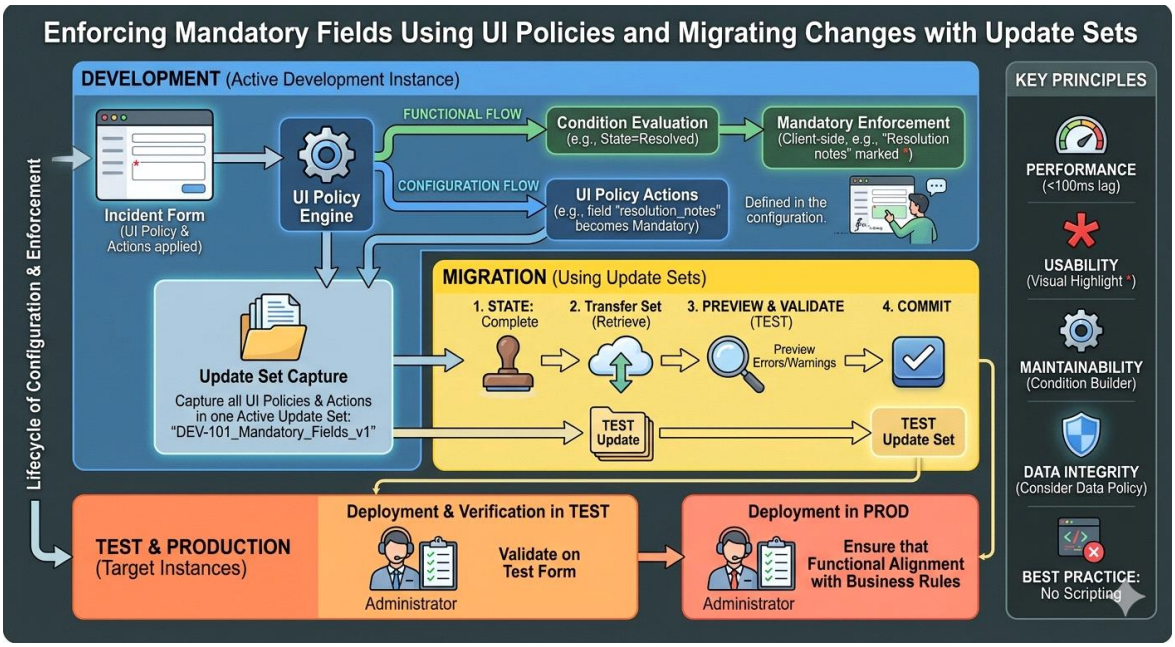
Deployment Workflow

Step	Action	Requirement
1. State Management	Complete the Set	The Update Set must be marked as "Complete" before it can be retrieved by the target instance.
2. Previewing	Conflict Check	The administrator must "Preview" the set on the target instance to check for collisions or missing dependencies.
3. Commitment	Finalize	The set must be "Committed" only after all preview errors/warnings are resolved.

4. Best Practices Summary

Note: If you are making a field mandatory that is critical for the database (even outside the UI), convert the **UI Policy** to a **Data Policy**. This ensures the "Mandatory" rule is enforced at the server level, covering API calls and Excel imports.

- **Avoid Scripting:** Use the `Mandatory` dropdown in UI Policy Actions instead of `g_form.setMandatory()`.
- **Test Before Migration:** Always impersonate an end-user in the Development instance to verify the UI Policy triggers correctly before closing your Update Set.
- **Batching:** If moving multiple related changes, consider using **Update Set Batching** to simplify the migration of parent/child sets.


Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Lifecycle of Configuration & Enforcement
DEVELOPMENT (Active Development Instance)
Incident Form
(UI Policy & Actions applied)
UI Policy Engine
FUNCTIONAL FLOW
Condition Evaluation
(e.g., State=Resolved)
Mandatory Enforcement
(Client-side, e.g., "Resolution notes" marked *)
CONFIGURATION FLOW
UI Policy Actions
(e.g., field "resolution_notes" becomes Mandatory)
Defined in the configuration.
Update Set Capture
Capture all UI Policies & Actions in one Active Update Set: "DEV-101_Mandatory_Fields_v1"
MIGRATION (Using Update Sets)
1. STATE: Complete
2. Transfer Set (Retrieve)
3. PREVIEW & VALIDATE (TEST)
Preview Errors/Warnings
4. COMMIT
TEST Update
TEST Update Set
TEST & PRODUCTION (Target Instances)
Deployment & Verification in TEST
Administrator
Validate on Test Form
Deployment in PROD
Administrator
Ensure that Functional Alignment with Business Rules
KEY PRINCIPLES
PERFORMANCE
(<100ms lag)
USABILITY
(Visual Highlight *)
MAINTAINABILITY
(Condition Builder)
DATA INTEGRITY
(Consider Data Policy)
BEST PRACTICE:
No Scripting